



INTERCOMMUNICATION OF MICROSERVICES

Faculty of Technology - University of Sri Jayewardenepura

Hansini Gunasena

Visiting Lecturer

Senior Software Engineer – Virtusa Pvt Ltd

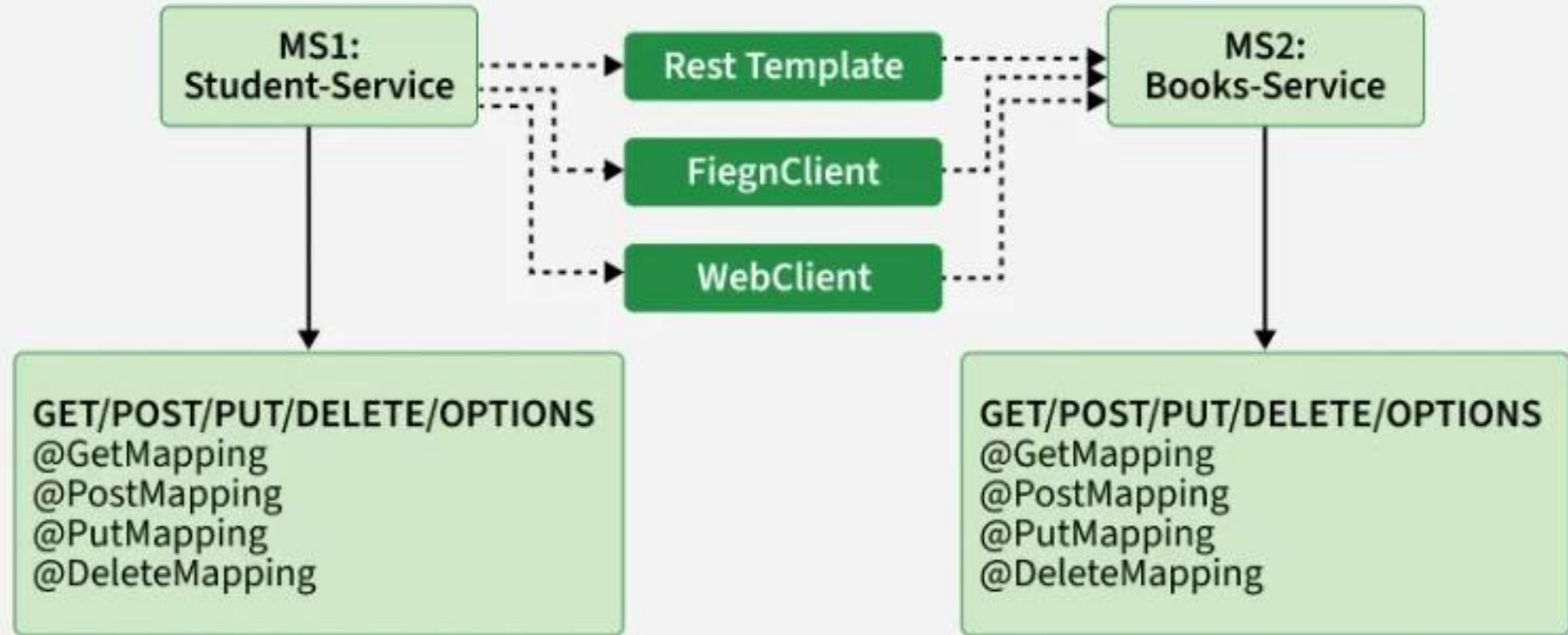
BSc In Computer science

Why Communication Between Microservices Matters

Microservices architecture breaks a large application into small, independent services that work together. These services must communicate to exchange data and perform business operations. Inter-Service Communication is the backbone of microservices, ensuring services remain loosely coupled yet fully functional as part of the larger system.

Each service owns its data and must share it to fulfill business needs.

Enables collaboration, scalability, flexibility, and reusability.

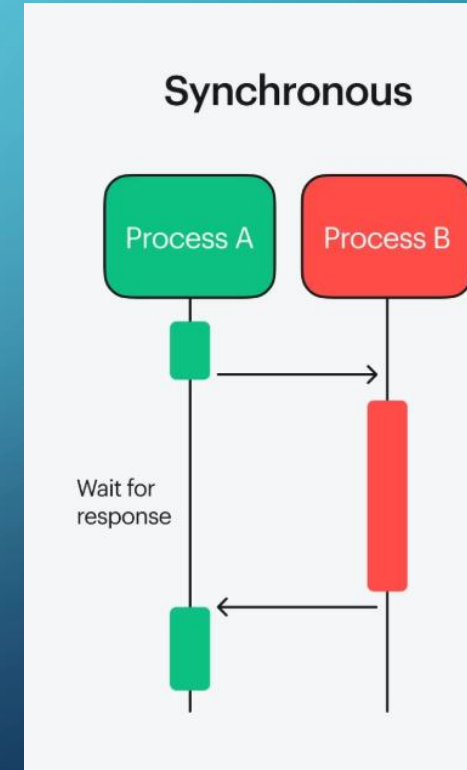
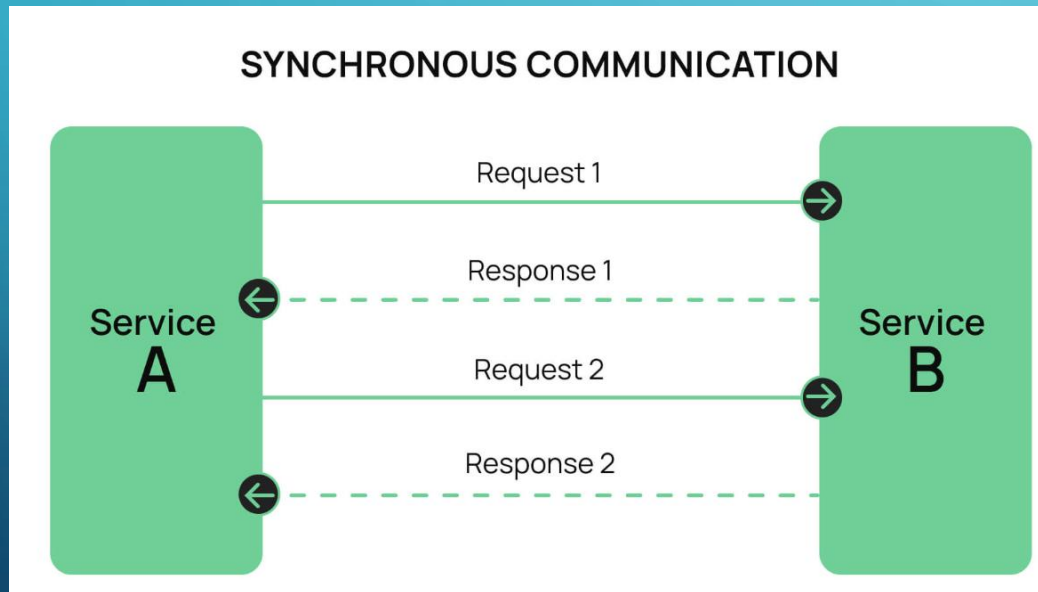


TYPES OF INTER- SERVICE COMMUNICATION

1. Synchronous Communication
2. Asynchronous Communication
3. Hybrid Communication

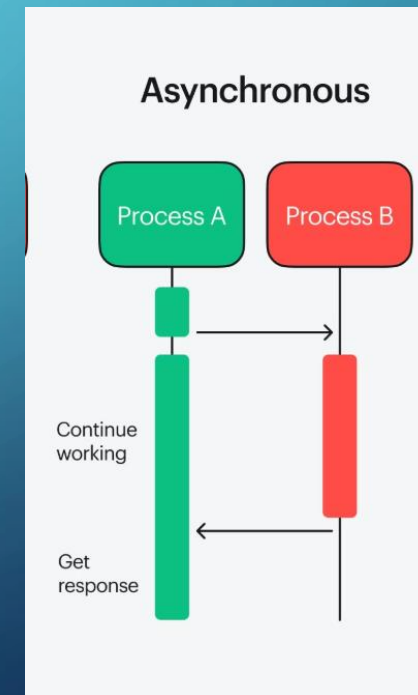
Synchronous Communication

- Real-time, request-response communication.
- Commonly implemented via REST APIs (HTTP), gRPC or GraphQL.
- Example: Student-Service calls Books-Service using REST to fetch available books.



ASYNCHRONOUS COMMUNICATION

- Services communicate without waiting for an immediate response.
- Implemented using Message Queues (RabbitMQ, Kafka, ActiveMQ).
- Example: Student-Service publishes an event "Student Registered" and Books-Service consumes it later.



HYBRID COMMUNICATION

- Combination of synchronous and asynchronous methods.
- Used when certain operations require real-time responses, while others can be event-driven.
- **Example:**
 - Student-Service sends a synchronous REST request to Books-Service to fetch a list of available books.
 - At the same time, Student-Service publishes an event asynchronously (Student Borrowed Book) so other services (like Notification-Service or Analytics-Service) can react later.

REST TEMPLATE

- **RestTemplate** is a Java class in Spring used to perform **HTTP requests** (GET, POST, PUT, DELETE) and consume RESTful web services.

It works like a simple client that your microservice uses to call another microservice's endpoint.

Why Do We Use RestTemplate?

- To call another microservice using **HTTP**
- To send or receive JSON/XML data
- To fetch resources from external APIs
- To integrate with legacy HTTP-based systems

HOW REST TEMPLATE WORKS (SIMPLE FLOW)

```
Order Service -----> makes REST call -----> Inventory Service  
      (uses RestTemplate)
```

RestTemplate sends an HTTP request → waits for the response → returns the data.
This is **synchronous communication** (blocking).

HTTP Method	RestTemplate Method	Description	Example
GET	getForObject(url, responseType)	Returns only the response body	rest.getForObject(url, String.class)
GET	getForEntity(url, responseType)	Returns full HTTP response (status + headers + body)	rest.getForEntity(url, String.class)
POST	postForObject(url, request, responseType)	Sends POST request and returns response body	rest.postForObject(url, data, String.class)
POST	postForEntity(url, request, responseType)	Sends POST request and returns full response	rest.postForEntity(url, data, String.class)
POST	postForLocation(url, request)	Sends POST and returns the URI of the created resource	rest.postForLocation(url, data)
PUT	put(url, request)	Sends PUT request to update a resource	rest.put(url, updatedData)
DELETE	delete(url)	Deletes a resource	rest.delete(url)
PATCH	patchForObject(url, request, responseType)	Partially updates a resource (if API supports PATCH)	rest.patchForObject(url, patchData, String.class)
ANY (custom request)	exchange(url, HttpMethod, requestEntity, responseType)	Fully customizable; allows headers, body, method	rest.exchange(url, HttpMethod.GET, entity, String.class)
HEAD	headForHeaders(url)	Retrieves HTTP headers only	rest.headForHeaders(url)
OPTIONS	optionsForAllow(url)	Returns allowed HTTP methods for a URL	rest.optionsForAllow(url)



Example Usage

java

```
@RestController
public class OrderController {

    private final RestTemplate restTemplate;

    public OrderController(RestTemplate restTemplate) {
        this.restTemplate = restTemplate;
    }

    @GetMapping("/order/{id}")
    public String getOrderDetails(@PathVariable String id) {

        String inventoryUrl = "http://inventory-service/product/" + id;

        String productDetails =
            restTemplate.getForObject(inventoryUrl, String.class);

        return "Order details with product: " + productDetails;
    }
}
```

- Spring recommends using **WebClient** instead of RestTemplate for reactive programming and better performance.
- However, **RestTemplate** is still widely used in microservices because it is simple and reliable.
- **RestTemplate** = A simple Spring tool to call **REST APIs** from one service to another using **HTTP** (synchronous communication).

✓ GET Example

java

```
String response = restTemplate.getForObject("http://service/api/data", String.class);
```

✓ POST Example

java

```
MyRequest body = new MyRequest();  
MyResponse response =  
    restTemplate.postForObject("http://service/api/new", body, MyResponse.class);
```

HOW REST TEMPLATE HANDLES HTTP STATUS CODES

● Success Codes

Code	RestTemplate Behavior
200 OK	Returns response body
201 Created	Returns location / response
204 No Content	Returns without body

● Error Codes → RestTemplate Throws Exceptions

Code	Exception Thrown
400 Bad Request	<code>HttpClientErrorException.BadRequest</code>
404 Not Found	<code>HttpClientErrorException.NotFound</code>
409 Conflict	<code>HttpClientErrorException.Conflict</code>
500 Internal Server Error	<code>HttpServerErrorException.InternalServerError</code>
503 Service Unavailable	<code>HttpServerErrorException.ServiceUnavailable</code>

★ Most Useful 300-Range Codes for Microservices

Code	Name	Meaning / When Used
300 Multiple Choices	Multiple possible responses	Rarely used; the server has more than one option
301 Moved Permanently	Resource permanently moved to a new URL	Clients should update URL; common in API version migrations
302 Found (Temporary Redirect)	Temporary redirection	Resource moved temporarily; client should use the new URL for now
303 See Other	Redirect using GET	Often used after a POST to redirect to another resource
304 Not Modified	No need to resend resource	Used for caching; client can use the cached version
307 Temporary Redirect	Same as 302 but preserves HTTP method	POST stays POST, not changed to GET
308 Permanent Redirect	Same as 301 but preserves HTTP method	POST stays POST; used for permanent API changes

How It Relates to Microservices & RestTemplate

- 3xx codes are less common **between microservices**, but they matter for **API gateways, caching, and versioning**.
- RestTemplate **automatically follows redirects** by default.
- Example:
- If a microservice endpoint moves from **/v1/order** → **/v2/order**:
 - API Gateway may return **301** or **308**
 - RestTemplate will follow the new URL



Example: Handling Errors in RestTemplate

java

```
try {  
    ResponseEntity<String> response =  
        restTemplate.getForEntity("http://service/data", String.class);  
  
    return response.getBody();  
  
} catch (HttpClientErrorException | HttpServerErrorException ex) {  
    System.out.println("Status Code: " + ex.getStatusCode());  
    System.out.println("Error Body: " + ex.getResponseBodyAsString());  
}
```



RestTemplate Example with Redirects

java

```
ResponseEntity<String> response =  
    restTemplate.getForEntity("http://api/v1/order/10", String.class);  
  
System.out.println(response.getStatusCode());
```

If server returns **301**, RestTemplate will call the new URL automatically.

FEIGN CLIENT (SPRING CLOUD FEIGN)

Why Feign Client Is Used?

- To call another microservice without writing boilerplate HTTP code
- To make REST calls look like normal Java method calls
- To reduce code complexity in microservices
- To integrate easily with Ribbon, Eureka, or Spring Cloud Gateway

Simple Example

Without Feign (manual HTTP call)

java

 Copy code

```
RestTemplate restTemplate = new RestTemplate();  
String user = restTemplate.getForObject("http://USER-SERVICE/users/1", String.class);
```

With Feign Client — much simpler

java

 Copy code

```
@FeignClient(name = "user-service")  
public interface UserClient {  
    @GetMapping("/users/{id}")  
    UserDto getUserById(@PathVariable("id") Long id);  
}
```

Then you can simply inject and call it:

java

 Copy code

```
@Autowired  
private UserClient userClient;  
  
UserDto user = userClient.getUserById(1L);
```

★ How It Works

- You define a Java interface
- Feign automatically creates the implementation
- The client sends HTTP requests to the target service
- Works perfectly with service discovery, load balancing, and circuit breakers

Typical Use Case

Order Service calls User Service to get user details:

```
sql
```

```
Order Service → Feign Client → User Service
```

Benefits of Feign Client

Benefit	Explanation
Declarative	Just create an interface, Feign handles the rest
Cleaner code	No need for RestTemplate or WebClient code
Load balancing	Works with Spring Cloud LoadBalancer
Automatic service discovery	Works with Eureka (if used)
Supports interceptors	Logging, auth headers, tokens

When NOT to use Feign

- High-traffic systems needing ultra-low latency (gRPC is better)
- Workloads requiring asynchronous messaging (Kafka / RabbitMQ)
- When calling unreliable services (use circuit breakers)

WEB CLIENT?

WebClient is a modern, non-blocking, reactive HTTP client introduced in **Spring WebFlux**. It replaces the old **RestTemplate** and is recommended by Spring for all new applications.

It supports both:

- **Synchronous (blocking)** calls
- **Asynchronous (non-blocking)** reactive calls

Why WebClient Is Used

- To call other microservices or external APIs
- To perform **non-blocking** HTTP requests
- To handle **high-concurrency** workloads efficiently
- To replace RestTemplate (which is deprecated)

Key Features

- ✓ Non-blocking and reactive
- ✓ Supports streaming
- ✓ Lightweight and fast
- ✓ Works in both WebFlux and Spring MVC projects
- ✓ Supports filters, interceptors, and authentication headers



Basic Example (Simple GET Call)

java

```
WebClient webClient = WebClient.create("http://user-service");
```

```
UserDto user = webClient.get()  
    .uri("/users/{id}", 1)  
    .retrieve()  
    .bodyToMono(UserDto.class)  
    .block(); // blocking (optional)
```

Asynchronous (Non-Blocking) Example

java

```
Mono<UserDto> userMono = webClient.get()
    .uri("/users/{id}", 1)
    .retrieve()
    .bodyToMono(UserDto.class); // returns immediately

userMono.subscribe(user -> {
    System.out.println("Received user: " + user);
});
```

POST Request Example

java

```
UserDto saved = webClient.post()
    .uri("/users")
    .bodyValue(newUser)
    .retrieve()
    .bodyToMono(UserDto.class)
    .block();
```



Building a WebClient Bean (Best Practice)

java

```
@Configuration
public class WebClientConfig {

    @Bean
    public WebClient webClient(WebClient.Builder builder) {
        return builder.baseUrl("http://user-service").build();
    }
}
```



When to Use WebClient?

Use Case	Why WebClient Helps
High-traffic microservices	Non-blocking, better performance
Calling external APIs	Clean and reactive
Need streaming or large payloads	Supports reactive streaming
Migrating from RestTemplate	WebClient is the modern replacement



⚠ When NOT to Use Web Client

- If you need a simple declarative client → use **Feign Client**
 - For internal synchronous service-to-service calls → Feign is easier
 - If your team is unfamiliar with reactive programming → keep it simple
- 
- 
- 

★ What is Reactive Programming?

- **Reactive Programming** is a programming style where the system reacts to data changes, events, or streams *as they happen*, instead of waiting for tasks to finish one by one.
- It is **non-blocking, asynchronous**, and designed for **high-performance, scalable** applications.

Use it when:

- ✓ High traffic
- ✓ Real-time updates
- ✓ Many API/database calls
- ✓ Scaling microservices

"Reactive programming makes your application fast, non-blocking, and able to handle massive workloads efficiently."

9 Essential Components of Production Microservice App

ByteByteGo

